Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Analisis Kode Mesin, Disassembly, dan Optimasi

Pertemuan 15

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 4.3

Menganalisis kode mesin hasil *disassembly* dan menerapkan strategi optimasi program pada level instruksi.

CPMK yang didukung: CPMK04 ▪ **CPL yang didukung: CPL03**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan pengertian *disassembly* dan perbedaannya dengan *assembly*;
2. membongkar kode mesin dengan perintah `U` pada DEBUG serta perkakas sejenis;
3. menyebutkan keterbatasan *disassembly* dan mengenali pola hasilnya;
4. menghitung empat metrik optimasi serta menjelaskan bahwa tanpa pengukuran optimasi hanyalah dugaan;
5. menerapkan teknik optimasi level instruksi: *strength reduction*, `LEA`, *loop unrolling*, dan `XOR reg, reg`;
6. menjelaskan trade-off optimasi ukuran dan optimasi kecepatan.

Peta Materi

1. **Disassembly: Mengembalikan Kode Mesin ke Mnemonic**
   pengertian *disassembly*; perintah `U` pada DEBUG; perkakas sejenis; empat keterbatasan *disassembly*

2. **Mengenali Pola Hasil Disassembly dan Metrik Optimasi**
   sembilan pola yang sering muncul; empat metrik optimasi; tabel perkiraan siklus; tanpa pengukuran optimasi hanyalah dugaan

3. **Strategi Optimasi pada Level Instruksi**
   register daripada memori; *strength reduction*; pergeseran pengganti `MUL`/`DIV`; `LEA`; *unrolling*; `XOR reg, reg`; trade-off; pengukuran yang adil; etika optimasi

Praktik pada pertemuan ini

Membongkar `naif.asm`, `teroptimasi.asm`, dan `teroptimasi-unroll.asm` dengan perintah `U`, menghitung metriknya, lalu menulis `optimasi-anda.asm` beserta laporan pengukuran.

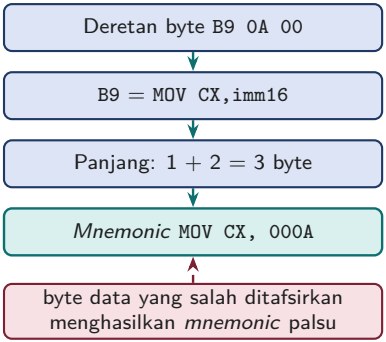
BAGIAN

Disassembly: Mengembalikan Kode Mesin ke Mnemonic

Mengapa Disassembly Diperlukan

- **Analisis perangkat lunak pihak ketiga.** Bila kode sumber tidak tersedia, satu-satunya cara memahami perilaku program adalah membongkar kode mesinnya.
- **Analisis keamanan.** Program mencurigakan dibongkar untuk menemukan instruksi yang berbahaya.
- **Verifikasi hasil kompilasi.** Memastikan kompilator benar-benar menghasilkan instruksi yang dikehendaki.
- **Optimasi terarah.** Hambatan yang sesungguhnya harus ditemukan pada kode mesin, bukan pada kode sumber.

Sumber: Bab XVI, Mengapa Disassembly Diperlukan



Perintah U pada DEBUG

U (*unassemble*) membaca sejumlah byte mulai dari alamat tertentu, menerjemahkannya menjadi *mnemonic*, lalu menampilkan alamat, byte mesin, dan instruksinya berdampingan.

```
-U 100 108
1D2F:0100 B90A00    MOV   CX,000A
1D2F:0103 BE0000    MOV   SI,0000
1D2F:0106 B80000    MOV   AX,0000
```

```
U               ; 32 byte dari CS:IP
U 100 108       ; offset 100h-108h
U 100 L 8       ; 8 byte dari 100h
```

Byte mesin dicetak lebih dahulu karena itulah data yang tersimpan di memori; *mnemonic* adalah penafsiran DEBUG atas byte tersebut.

Sumber: Bab XVI, Perintah U pada DEBUG dan Perkakas Sejenis

Perkakas Disassembly untuk Program 8086

Perkakas	Perintah	Kegunaan utama
MS-DEBUG	`U [alamat] [L n]`	Bongkar kode di dalam memori; tersedia di DOSBox tanpa pemasangan tambahan
Emu8086	tombol `Disassembler`	Kode mesin dan *mnemonic* berdampingan dengan tampilan register dan memori
GUI Turbo Assembler	menu `Disassemble`	Membongkar berkas `.exe` lengkap dengan navigasi prosedur
`ndisasm` (NASM)	`ndisasm -b 16 prog.com`	Membongkar berkas biner secara massal; keluarannya berupa teks yang dapat disalin
`objdump` (GNU)	`objdump -d -M intel prog.o`	Membongkar berkas objek hasil kompilator; berguna pada arsitektur modern

Sumber: Bab XVI, Perkakas Disassembly untuk Program 8086

Empat Keterbatasan Disassembly

- **Kode dan data sulit dibedakan.** Byte `48h` dapat berarti instruksi `DEC AX` atau huruf `'H'`; *disassembler* bekerja dengan asumsi semua byte adalah instruksi.
- **Alamat relatif dan displacement.** `LOOP 0109` di dalam memori sesungguhnya berisi byte `E2h F7h`, yaitu "mundur 9 byte"; hasil hitungan ulang hanya benar bila instruksi berada di posisi yang tepat.
- **Nama simbolik hilang.** Variabel `larik` dan label `ULANG` tidak tersimpan di berkas biner; keduanya muncul sebagai `[SI+01F4]`.
- **Sulit dibaca dan dipelihara.** Komentar hilang, makro sudah dijabarkan, dan tidak ada satu pun nama yang bermakna.

Penting: Bongkar hanya daerah yang benar-benar berisi kode — periksa batas segmen dengan mencatat `CS` dan `IP` saat program dimulai.

Sumber: Bab XVI, Keterbatasan Disassembly

Rangkuman Bagian 1

- *Disassembly* mengubah kode mesin kembali menjadi *mnemonic*. Pekerjaannya searah dan tidak dapat membalikkan *assembly* sepenuhnya, karena nama simbolik, komentar, dan batas data hilang saat perakitan.
- Perintah `U` pada DEBUG menampilkan alamat, byte mesin, dan *mnemonic* sekaligus; byte mesin adalah acuan yang benar.
- Perkakas lain: *disassembler* Emu8086, GUI Turbo Assembler, `ndisasm`, dan `objdump`.
- Keterbatasan utama: kode dan data sulit dibedakan, *displacement* menggantikan nama variabel, dan semua penanda yang berguna bagi manusia hilang.

BAGIAN

Mengenali Pola Hasil Disassembly dan Metrik Optimasi

Pola Disassembly: Alur Kendali dan Prosedur

Pola kode sumber	Bentuk hasil *disassembly*	Cara mengenalinya
Pembandingan dan percabangan	`CMP AX,0064h` + `JE 0140h`	`CMP` atau `TEST` selalu diikuti lompatan bersyarat
Perulangan berpenghitung	`LOOP 0109h` atau `DEC CX` + `JNZ`	Lompatan mundur yang dikendalikan `CX`
Perulangan uji-di-akhir	`CMP SI,0014h` + `JB 0100h`	Lompatan mundur setelah pembandingan batas
Prologue prosedur	`PUSH BP` + `MOV BP,SP`	Menyimpan bingkai *stack* lama lalu menyediakan variabel lokal
Epilogue prosedur	`POP BP` + `RET`	`RET` selalu menjadi instruksi terakhir

Sumber: Bab XVI, Pola-Pola yang Sering Dijumpai

Pola Disassembly: Sistem dan Aritmatika

Pola kode sumber	Bentuk hasil *disassembly*	Cara mengenalinya
Inisialisasi segmen	`MOV AX,1D2Fh` + `MOV DS,AX`	Pemindahan ke register segmen selalu melalui register umum
Pemanggilan interupsi	`MOV AH,09h` + `INT 21h`	Nomor fungsi dimuat ke `AH` tepat sebelum `INT`
Perkalian pangkat dua	`SHL AX,1` atau `SHL AX,CL`	Pergeseran menggantikan `MUL`; tidak ada operand pengali
Pengosongan register	`XOR AX,AX`	Kedua operand sama; hasilnya selalu nol

Lompatan mundur adalah tanda perulangan yang paling dapat diandalkan, sedangkan lompatan maju yang jauh biasanya menandai percabangan pemilihan.

Tiga Pola dalam Satu Potongan Hasil Disassembly

Perhatikan bahwa `LOOP 0108` menunjuk ke alamat `0108h`, lebih kecil daripada alamat dirinya sendiri (`010Dh`).

```
1D2F:0100 B82F1D     MOV   AX,1D2F
1D2F:0103 8ED8       MOV   DS,AX
1D2F:0105 B90A00     MOV   CX,000A
1D2F:0108 0304       ADD   AX,[SI]
1D2F:010A 83C602     ADD   SI,+02
1D2F:010D E2F9       LOOP  0108
1D2F:010F B409       MOV   AH,09
1D2F:0111 BA1001     MOV   DX,0110
1D2F:0114 CD21       INT   21
```

Tiga pola sekaligus

- Inisialisasi segmen: `MOV AX,1D2F` lalu `MOV DS,AX`.
- Perulangan berpenghitung `CX`: `LOOP 0108`.
- Pemanggilan interupsi: `MOV AH,09` lalu `INT 21`.

Sumber: Bab XVI, Pola-Pola yang Sering Dijumpai

Empat Metrik Optimasi

1. **Jumlah instruksi.** Banyaknya instruksi yang *dieksekusi*, bukan yang tertulis di kode. Metrik ini diperoleh dengan menghitung iterasi.
2. **Perkiraan jumlah siklus.** Dihitung dari tabel siklus tiap instruksi, bukan diukur langsung pada mesin.
3. **Ukuran kode dalam byte.** Panjang kode mesin yang dihasilkan; penting pada sistem tertanam dengan memori terbatas.
4. **Jumlah akses memori.** Akses memori jauh lebih mahal daripada akses register, sehingga pengurangan satu akses saja sudah berarti.

Ukuran keberhasilan ditetapkan lebih dahulu

Sebelum memperbaiki sesuatu, metrik yang dipakai harus dipilih agar perbandingan sebelum dan sesudah dapat dipercaya.

Sumber: Bab XVI, Metrik Optimasi

Perkiraan Siklus Instruksi 8086 (1)

Instruksi	Byte	Siklus (perkiraan)	Keterangan
`MOV reg, reg`	2	2	Paling murah
`MOV reg, imm16`	3	4	Memuat konstanta
`ADD reg, reg`	2	3	Aritmatika register
`XOR reg, reg`	2	3	Hasilnya sama dengan `MOV reg,0`, tetapi lebih pendek
`ADD AX, [SI]`	2	$9 + 5 = 14$	Alamat efektif `[SI]` = 5 siklus
`ADD AX, [SI+2]`	3	$9 + 8 = 17$	EA dengan *displacement* lebih mahal

Sumber: Bab XVI, Tabel Perkiraan Siklus Instruksi 8086

Perkiraan Siklus Instruksi 8086 (2)

Instruksi	Byte	Siklus (perkiraan)	Keterangan
`MOV mem, reg`	4	$9 + 6 = 15$	Alamat langsung
`SHL reg, 1`	2	2	Pergeseran satu bit
`SHL reg, CL`	2	$8 + 4{\cdot}\mathrm{CL}$	Naik seiring nilai `CL`
`LEA reg, [SI+2]`	3	$2 + 8 = 10$	Tidak mengubah *flag*; lebih ringkas daripada `MOV` + `ADD`
`MUL reg16`	2	118–133	Sangat mahal
`DIV reg16`	2	$\approx$150	Paling mahal
`LOOP label`	2	17 / 5	17 siklus bila melompat, 5 siklus bila berhenti

Angka berlaku pada kondisi ideal: tanpa *wait state*, operand di register, dan *prefetch queue* selalu terisi.

Tanpa Pengukuran, Optimasi Hanyalah Dugaan

- **Hambatan yang berbeda.** Perbaikan hanya berguna bila menyerang bagian yang paling banyak dieksekusi; mempercepat instruksi yang dijalankan seratus kali tidak berarti apa-apa bila ada perulangan yang berjalan sejuta kali.
- **Metrik yang bertentangan.** Suatu perubahan dapat mempercepat program tetapi memperbesar kode; bila target program adalah ukuran, perubahan itu justru merugikan.
- **Lingkungan yang berbeda.** Hasil yang cepat pada emulator belum tentu cepat pada perangkat keras sesungguhnya, dan sebaliknya.

Aktivitas

Tuliskan dugaan Anda tentang bagian program yang paling lambat, bongkar program itu dengan `U`, lalu bandingkan dugaan dengan kenyataan pada jurnal praktikum.

Sumber: Bab XVI, Tanpa Pengukuran, Optimasi Hanyalah Dugaan

Rangkuman Bagian 2

- Pola hasil *disassembly* dapat dikenali: pembandingan dan percabangan, perulangan, *prologue*/*epilogue*, inisialisasi segmen, dan pemanggilan interupsi.
- Ciri perulangan yang paling andal adalah lompatan mundur ke alamat yang lebih kecil daripada alamat instruksi lompatan.
- Empat metrik optimasi: jumlah instruksi yang dieksekusi, perkiraan siklus, ukuran kode dalam byte, dan jumlah akses memori.
- Jarak biaya antarinstrusi sangat lebar: `MOV reg, reg` 2 siklus, sedangkan `DIV reg16` sekitar 150 siklus.
- Angka perkiraan hanya berlaku pada kondisi ideal; tanpa pengukuran pada mesin yang dipakai, optimasi hanyalah dugaan.

BAGIAN

Strategi Optimasi pada Level Instruksi

Memilih Register daripada Memori

Teknik paling mendasar adalah menyimpan nilai yang sering dipakai di dalam register, bukan di memori.

```
    ; --- kurang baik: nilai diambil dari memori setiap iterasi ---
    MOV CX, 10
ULANG:
    ADD AX, jumlah      ; akses memori: 9 + 6 = 15 siklus
    LOOP ULANG

    ; --- lebih baik: nilai sudah ada di register ---
    MOV CX, 10
    MOV BX, jumlah      ; sekali saja: 8 + 6 = 14 siklus
ULANG2:
    ADD AX, BX          ; akses register: 3 siklus
    LOOP ULANG2
```

Sepuluh iterasi versi pertama $10 \times 15 = 150$ siklus; versi kedua $14 + 10 \times 3 = 44$ siklus.

Sumber: Bab XVI, Memilih Register daripada Memori

Strength Reduction: Penggantian Operasi

Operasi asal	Pengganti	Syarat berlaku
`MUL BX`, `BX` $= 2$	`ADD AX, AX`	Pengali tepat dua; hasil tidak memerlukan `DX`
`MUL BX`, `BX` $= 2^n$	`SHL AX, n`	Pengali pangkat dua dan tak bertanda
`DIV BX`, `BX` $= 2^n$	`SHR AX, n`	Pembagi pangkat dua, nilai tak bertanda, sisa tidak diperlukan
`IDIV BX`, `BX` $= 2^n$	`SAR AX, n`	Nilai bertanda (komplemen dua); pembulatan ke bawah
`MOV AX, 0`	`XOR AX, AX`	Hasil harus nol; *flag* memang boleh berubah
`CMP AX, 0`	`TEST AX, AX`	Uji dilakukan hanya terhadap nol
`ADD SI, 2`	`LEA SI, [SI+2]`	*Flag* harus dipertahankan agar pembandingan sebelumnya tetap sahih
`DEC CX` + `JNZ`	`LOOP`	Penghitung di `CX` dan jarak lompatan kurang dari ± 128 byte

Sumber: Bab XVI, Strength Reduction

Pergeseran sebagai Pengganti MUL dan DIV

MUL BX memerlukan 118–133 siklus, sedangkan SHL AX, 1 hanya 2 siklus. Untuk mengalikan dengan delapan, tiga kali pergeseran satu bit sudah cukup.

```
; --- varian cepat: 3 instruksi, 6 byte, 6 siklus ---
SHL AX, 1           ; AX = AX x 2
SHL AX, 1           ; AX = AX x 4
SHL AX, 1           ; AX = AX x 8

; --- varian kecil: 2 instruksi, 4 byte, 20 siklus ---
MOV CL, 3           ; CL = banyak pergeseran
SHL AX, CL          ; 8 + 4 x 3 = 20 siklus
```

Syarat yang tidak boleh dilupakan

Pergeseran hanya setara perkalian bila nilai yang digeser tak bertanda dan bit yang keluar boleh hilang. MUL memerhatikan CF dan OF untuk mendeteksi luapan, SHL tidak.

LEA untuk Aritmatika Tanpa Mengubah *Flag*

ADD mengubah ZF sehingga lompatan bersyarat menjadi salah; LEA menghitung alamat tanpa menyentuh *flag*.

```
CMP AX, 0           ; ZF = 1 bila AX = 0
ADD SI, 2           ; SALAH: ADD mengubah ZF
JE  kosong          ; lompatan tidak lagi dapat dipercaya

CMP AX, 0           ; ZF = 1 bila AX = 0
LEA SI, [SI+2]      ; BENAR: LEA tidak mengubah flag
JE  kosong          ; lompatan masih sahih
```

LEA SI, [BX+SI+4] menjumlahkan BX, SI, dan konstanta 4 sekaligus dalam satu instruksi. Namun LEA bukan pilihan yang lebih cepat: pada 8086 ia 10 siklus, sedangkan ADD SI, 2 hanya 4 siklus — keunggulannya keringkasan dan sifat tidak mengubah *flag*.

Sumber: Bab XVI, LEA untuk Aritmatika Tanpa Mengubah Flag

Mengurangi Lompatan dan *Loop Unrolling*

LOOP memerlukan 17 siklus dan menghalangi *prefetch queue* mengisi instruksi berikutnya. *Unrolling* menggandakan isi perulangan sehingga overhead LOOP berkurang separuh.

```
    ; --- perulangan biasa: 1 elemen per iterasi ---
    MOV CX, 10
ULANG:
    ADD AX, [SI]
    ADD SI, 2
    LOOP ULANG

    ; --- setelah unrolling 2x: 2 elemen per iterasi ---
    MOV CX, 5           ; cukup 5 iterasi
ULANG2:
    ADD AX, [SI]        ; elemen ke-1
    ADD AX, [SI+2]      ; elemen ke-2
    ADD SI, 4           ; maju dua word
    LOOP ULANG2
```

Harganya jelas: kode menjadi lebih panjang.

Mengosongkan Register dengan `XOR reg, reg`

Cara ini benar dan mudah dibaca, tetapi bukan yang terpendek.

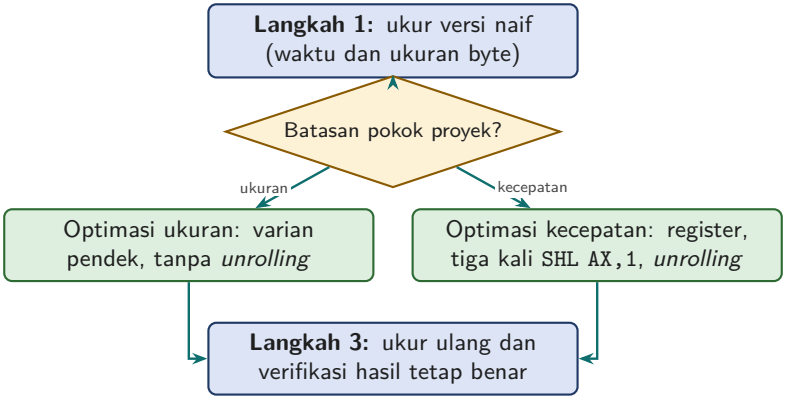
```
MOV AX, 0          ; 3 byte (B8 00 00), 4 siklus
XOR AX, AX         ; 2 byte (31 C0),    3 siklus
SUB AX, AX         ; 2 byte (29 C0),    3 siklus
```

`XOR AX, AX` lebih pendek satu byte dan lebih cepat satu siklus karena tidak perlu memuat nilai segera; ia juga mengosongkan register 16 bit sekaligus.

Harganya adalah *flag*

`XOR` mengubah `ZF`, `SF`, `PF` serta menghapus `CF` dan `OF`. Bila dipakai di antara sebuah pembandingan dan lompatan bersyaratnya, program justru rusak — tempatkan pada awal blok, bukan di tengah alur keputusan.

Trade-off: Optimasi Ukuran atau Kecepatan



Tidak ada optimasi yang gratis: setiap perubahan hampir selalu memindahkan biaya dari satu metrik ke metrik lain.

Sumber: Bab XVI, Trade-off: Ukuran Kode atau Kecepatan

Pengukuran yang Adil

1. **Masukan sama dan jumlah iterasi sama.** Kedua versi memakai larik yang sama dan mengerjakan pekerjaan yang sama banyaknya; penghitung iterasi dicatat untuk membuktikannya.
2. **Batas pengukuran sama.** Waktu mulai dicatat tepat sebelum pekerjaan dan berhenti tepat sesudahnya, tanpa pemuatan program, pencetakan keluaran, atau penekanan tombol.
3. **Diulang beberapa kali.** Jalankan minimal lima kali, buang percobaan pertama (pemanasan), lalu ambil nilai tengah.

```
MOV AH, 00h          ; baca pencacah tick BIOS
INT 1Ah              ; CX:DX = tick sejak tengah malam
MOV tick_awal, DX    ; simpan titik awal

; ... pekerjaan yang diukur ...

INT 1Ah              ; baca lagi
SUB DX, tick_awal    ; selisih tick (18,2 tick = 1 detik)
```

Sumber: Bab XVI, Pengukuran yang Adil

Etika dan Batas Optimasi

- **Benar lebih dahulu, cepat kemudian.** Program yang cepat tetapi salah tidak berguna; uji kebenaran setiap versi dengan masukan yang sama sebelum membandingkan kecepatannya.
- **Jangan mengoptimasi dini.** Optimasi hanya layak pada bagian yang sudah terbukti menjadi hambatan melalui pengukuran, bukan pada bagian yang hanya *diperkirakan* lambat.
- **Jaga keterbacaan dan dokumentasi.** Setiap trik diberi komentar yang menjelaskan alasan dan syarat berlakunya, dan versi naif disimpan sebagai pembanding.

Optimasi yang dapat dipertanggungjawabkan

Ada pengukuran sebelum, ada penjelasan mengapa perubahan dipilih, dan ada pengukuran sesudah. Bila salah satunya tidak ada, yang dikerjakan bukan optimasi melainkan sekadar perubahan.

Sumber: Bab XVI, Etika dan Batas Optimasi

Rangkuman Bagian 3

- Teknik paling mendasar adalah menyimpan nilai yang sering dipakai di register, bukan di memori: 150 siklus menjadi 44 siklus pada sepuluh iterasi.
- *Strength reduction* menggantikan operasi mahal dengan operasi murah, tetapi hanya sahih bila syaratnya dipenuhi — misalnya pengali harus pangkat dua dan nilai harus tak bertanda.
- `LEA` menghitung alamat dalam satu instruksi tanpa mengubah *flag*, sehingga pembandingan sebelumnya tetap sahih.
- *Unrolling* mengurangi separuh overhead `LOOP` dengan harga kode yang lebih panjang; `XOR reg, reg` lebih pendek satu byte dan lebih cepat satu siklus daripada `MOV reg, 0`.
- Optimasi ukuran dan optimasi kecepatan sering bertentangan; pilihan ditentukan batasan proyek dan dibuktikan dengan pengukuran yang adil.

Contoh 1: Versi Naif

Ketiga contoh mengerjakan pekerjaan sama: menjumlahkan sepuluh elemen larik $10, 20, \ldots, 100$, lalu mengalikan jumlahnya (550) dengan delapan sehingga hasilnya `1130h`.

```
    MOV CX, 10          ; 10 elemen
    MOV SI, 0           ; indeks byte
    MOV AX, 0           ; akumulator = 0
ULANG:
    ADD AX, larik[SI]   ; ambil dari memori
    ADD SI, 2           ; maju satu word
    LOOP ULANG
    MOV jumlah, AX      ; simpan ke memori
    MOV AX, jumlah      ; muat kembali lagi
    MOV BX, 8           ; pengali = 8
    MUL BX              ; 118-133 siklus
    MOV hasil, AX       ; simpan hasil akhir
```

Empat pemborosan

1. `ADD AX, larik[SI]` memakai *displacement* sehingga alamat efektifnya 8 siklus, bukan 5.
2. `MOV AX, 0` tiga byte, padahal ada cara dua byte.
3. Hasil ditulis ke memori lalu dibaca kembali — dua akses memori yang tidak perlu.
4. `MUL BX` untuk mengalikan delapan adalah pilihan paling mahal: 118–133 siklus.

`naif.asm`

Contoh 2: Versi Teroptimasi

Empat perbaikan diterapkan tanpa mengubah hasil akhir: `hasil` tetap bernilai `1130h`.

```
    MOV CX, 10          ; penghitung iterasi
    MOV SI, OFFSET larik ; SI ke larik
    XOR AX, AX          ; akumulator = 0
ULANG:
    ADD AX, [SI]        ; tanpa displacement
    ADD SI, 2           ; maju satu word
    LOOP ULANG
    SHL AX, 1           ; AX = AX x 2
    SHL AX, 1           ; AX = AX x 4
    SHL AX, 1           ; AX = AX x 8
    MOV hasil, AX       ; satu akses memori
```

Empat perbaikan

- `SI` menunjuk langsung ke larik, sehingga `ADD AX, [SI]` hanya 5 siklus alamat efektif.
- `XOR AX, AX` mengosongkan akumulator dalam dua byte dan tiga siklus.
- Nilai antara tidak lagi bolak-balik ke memori; `AX` dipakai sebagai akumulator sampai akhir.
- `MUL BX` diganti tiga `SHL AX, 1` — enam siklus, sekitar dua puluh kali lebih cepat.

`teroptimasi.asm`

Contoh 3: Teroptimasi dengan *Loop Unrolling*

Isi perulangan digandakan sehingga satu iterasi mengerjakan dua elemen; jumlah iterasi turun dari sepuluh menjadi lima.

```
    MOV CX, 5           ; cukup 5 iterasi
    MOV SI, OFFSET larik
    XOR AX, AX          ; akumulator = 0
ULANG:
    ADD AX, [SI]        ; elemen ke-1
    ADD AX, [SI+2]      ; elemen ke-2
    ADD SI, 4           ; maju dua word
    LOOP ULANG
    SHL AX, 1           ; AX = AX x 2
    SHL AX, 1           ; AX = AX x 4
    SHL AX, 1           ; AX = AX x 8
    MOV hasil, AX       ; satu akses memori
```

Yang diperoleh dan harganya

- Overhead `LOOP` (17 siklus) berkurang separuh karena iterasi hanya lima kali.
- Dua elemen diambil per iterasi: `ADD AX, [SI]` dan `ADD AX, [SI+2]`.
- Alamat maju empat byte sekaligus dengan `ADD SI, 4`.
- Harganya: ukuran kode bertambah tiga byte dibandingkan versi teroptimasi.

`teroptimasi-unroll.asm`

Perbandingan Hasil Disassembly: Naif vs Teroptimasi

Keduanya dibongkar dengan asumsi program dimuat pada alamat `1D2Fh:0100`, larik pada offset `01F4h`, `jumlah` pada `0208h`, dan `hasil` pada `020Ah`.

Versi naif

```
1D2F:0103 BE0000    MOV  SI,0000
1D2F:0106 B80000    MOV  AX,0000
1D2F:0109 0384F401 ADD  AX,[SI+01F4]
1D2F:0110 E2F7      LOOP 0109
1D2F:0112 89060802 MOV  [0208],AX
1D2F:0116 8B060802 MOV  AX,[0208]
1D2F:011A BB0800    MOV  BX,0008
1D2F:011D F7E3      MUL  BX
```

11 instruksi, 35 byte

Versi teroptimasi

```
1D2F:0103 BEF401    MOV  SI,01F4
1D2F:0106 31C0      XOR  AX,AX
1D2F:0108 0304      ADD  AX,[SI]
1D2F:010D E2F9      LOOP 0108
1D2F:010F D1E0      SHL  AX,1
1D2F:0111 D1E0      SHL  AX,1
1D2F:0113 D1E0      SHL  AX,1
1D2F:0115 89060A02 MOV  [020A],AX
```

10 instruksi, 25 byte

Nama simbolik `larik` hilang menjadi *displacement* `01F4h`; `MOV AX, 0000` (3 byte) menjadi `XOR AX, AX` (2 byte); `MUL BX` menjadi tiga `SHL AX, 1`. Hasil *disassembly* tidak memberi tahu *mengapa* perubahan itu dilakukan — penjelasannya hanya ada pada perbandingan metrik.

Tabel Perbandingan Hasil Pengukuran

Versi	Instruksi dieksekusi	Ukuran kode	Perkiraan siklus	Akses memori
Naif	38	35 byte	≈546	13
Teroptimasi	37	25 byte	≈370	11
Teroptimasi + *unrolling*	27	28 byte	≈280	11

Pelajaran dari tabel

Jumlah instruksi saja menyesatkan: naif 38 berbanding teroptimasi 37 hampir sama, padahal perkiraan siklusnya turun sekitar 32 persen. Yang menentukan adalah *jenis* instruksi dan banyaknya akses memori. Versi *unrolling* paling cepat, tetapi ukurannya bertambah tiga byte. Angka ini masih perkiraan — pembuktiannya tetap memerlukan pengukuran pada mesin.

Judul praktikum

Menganalisis, mengoptimasi, dan mengukur program pada level instruksi.
Sub-CPMK: 4.3 ▪ **Waktu:** 150 menit ▪ **Bentuk:** kelompok (2–3 mahasiswa) dan laporan individu

Alat dan bahan

- DOSBox dan MS-DEBUG, atau Emu8086
- TASM dan TLINK (atau GUI Turbo Assembler)
- Berkas `naif.asm`, `teroptimasi.asm`, dan `teroptimasi-unroll.asm`
- Lembar kerja pengukuran dan jurnal praktikum

Dikumpulkan: `optimasi-anda.asm`, tabel perbandingan *disassembly*, tabel pengukuran, dan analisis singkat.

Langkah kerja

1. Rakit dan jalankan ketiga versi; `hasil` harus bernilai `1130h`.
2. Bongkar dengan `U`; catat alamat, byte mesin, *mnemonic*, dan tandai polanya.
3. Tandai inefisiensi versi naif dan hitung perkiraan siklusnya.
4. Salin menjadi `optimasi-anda.asm`; terapkan minimal tiga teknik optimasi.
5. Ukur ketiga versi dengan `INT 1Ah`; lima kali, buang percobaan pertama.

Latihan

1. Jelaskan mengapa *disassembly* tidak dapat mengembalikan kode sumber asli. Sebutkan tiga informasi yang hilang selama perakitan dan mengapa informasi itu tidak tersimpan di dalam berkas `.exe`.

2. Byte `48h` dapat berarti instruksi `DEC AX` atau karakter `'H'`. Jelaskan bagaimana keadaan itu menyulitkan *disassembly*, lalu sebutkan dua cara praktis menentukan batas daerah kode dan daerah data.

3. Diberikan potongan hasil *disassembly* berisi `MOV CX,000A`, `XOR BX,BX`, `ADD AX,[SI]`, `ADD SI,+02`, dan `LOOP 0205`. Pola apa yang tampak, berapa kali `ADD AX, [SI]` dieksekusi, dan berapa jarak relatif yang disimpan pada byte `E2h F9h`?

4. Perulangan 20 kali berisi `ADD AX, [SI]`, `ADD SI, 2`, dan `LOOP`. Hitung perkiraan jumlah siklusnya, lalu sebutkan dua alasan mengapa angka itu belum tentu sama dengan waktu sesungguhnya.

5. Tuliskan hasil *strength reduction* untuk `MOV AX, 0`, `ADD AX, nilai`, `MOV BX, 16`, `MUL BX`, `CMP AX, 0`, dan `JE kosong`, beserta syarat sahihnya.

Rangkuman

- *Disassembly* mengubah kode mesin kembali menjadi *mnemonic*; pekerjaan ini tidak dapat membalikkan *assembly* sepenuhnya karena nama simbolik, komentar, dan batas data hilang saat perakitan.
- Perintah `U` pada DEBUG menampilkan alamat, byte mesin, dan *mnemonic* sekaligus; perkakas lain: Emu8086, GUI Turbo Assembler, `ndisasm`, dan `objdump`.
- Pola hasil *disassembly* dapat dikenali: pembandingan dan percabangan, perulangan, *prologue* dan *epilogue* prosedur, inisialisasi segmen, dan pemanggilan interupsi.
- Empat metrik optimasi: jumlah instruksi yang dieksekusi, perkiraan siklus, ukuran kode dalam byte, dan jumlah akses memori. Tanpa pengukuran, optimasi hanyalah dugaan.
- *Strength reduction* hanya sahih bila syaratnya dipenuhi. Optimasi ukuran dan kecepatan sering bertentangan, sehingga pilihan ditentukan batasan proyek.
- Batas etika optimasi: kebenaran lebih dahulu, larangan optimasi dini, serta kewajiban menjaga keterbacaan.

Cek Pemahaman

1. Perintah MS-DEBUG untuk membongkar kode mesin menjadi *mnemonic* adalah ...
   1.1 `T`
   1.2 `U`
   1.3 `D`
   1.4 `G`
2. Nama variabel `larik` berubah menjadi `[SI+01F4]` pada hasil *disassembly* karena ...
   2.1 nama simbolik tidak disimpan di berkas biner
   2.2 DEBUG selalu menghapus nama simbolik
   2.3 linker mengubahnya menjadi alamat relatif
   2.4 disassembler hanya membaca register
3. Instruksi tunggal pengganti `MUL BX` untuk mengalikan `AX` (tak bertanda) dengan delapan adalah ...
   3.1 `SHR AX, 3`
   3.2 `SHL AX, 3`
   3.3 `SAR AX, 3`
   3.4 `DIV AX, 8`

Penutup

Sebelum ujian akhir semester

- Selesaikan laporan praktikum 15: `optimasi-anda.asm`, tabel perbandingan *disassembly*, dan tabel pengukuran.
- Lengkapi jurnal dan refleksi: apakah dugaan Anda tentang bagian yang paling lambat terbukti setelah diukur?
- Telusuri ketiga berkas contoh satu instruksi demi satu untuk bagian yang belum dikuasai.
- Siapkan presentasi proyek akhir ($7 + 3$ menit); setiap klaim perbaikan wajib disertai angka.

Bacaan lanjutan: Bab XVI bagian Pengukuran yang Adil dan Etika dan Batas Optimasi; prosedur pengukuran pada Bab XV.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil